

1. Hooks de Rendimiento en React

¿Qué son los Hooks de Rendimiento?

Los **Hooks de Rendimiento** son funciones especializadas provistas por React para optimizar la velocidad y el consumo de recursos de una aplicación web. Su función primordial es evitar cálculos computacionales pesados e innecesarios o prevenir re-renderizados redundantes de componentes secundarios cuando el estado general de la aplicación cambia.

Los dos principales hooks de rendimiento integrados son:

- **useMemo:** Memoriza (*cachea*) el valor resultante del cálculo de una función costosa.
- **useCallback:** Memoriza la definición de una instancia de función entre renderizados para mantener su igualdad referencial.

Características principales

- **Basados en arreglos de dependencias:** Solo recalculan el valor o recrean la función si al menos una de las dependencias declaradas en su arreglo [...] cambia de valor entre renderizados.
- **Persistencia referencial:** Garantizan que los objetos o funciones conserven la misma referencia de memoria (===), evitando disparar efectos secundarios (useEffect) o re-renderizados de componentes optimizados con React.memo.
- **Costo de sobrecarga:** Implementar estos hooks conlleva un costo computacional mínimo por la memoria asignada para la caché y la comparación de dependencias; por ello, deben aplicarse selectivamente en operaciones pesadas o cuellos de botella reales, no en operaciones triviales.

¿Para qué sirven?

Sirven para mantener una interfaz gráfica fluida (60 FPS), eliminando bloqueos en el hilo principal de JavaScript (*main thread*) al filtrar colecciones masivas de datos, procesar transformaciones matemáticas complejas o pasar *callbacks* a componentes hijos dependientes.

```
import React, { useState, useMemo } from 'react';

function calculoPesado(numero) {
  console.log('Ejecutando cálculo intensivo...');
  let total = 0;
  for (let i = 0; i < 500000000; i++) {
    total += numero;
  }
  return total;
}
```

```

function OptimizadorRendimiento() {
  const [contador, setContador] = useState(5);
  const [modoOscuro, setModoOscuro] = useState(false);

  // useMemo evita que calculoPesado se ejecute al cambiar el tema
  (modoOscuro)
  const resultadoCalculado = useMemo(() => {
    return calculoPesado(contador);
  }, [contador]);

  const estiloTema = {
    backgroundColor: modoOscuro ? '#222' : '#fff',
    color: modoOscuro ? '#fff' : '#000',
    padding: '20px',
    borderRadius: '8px'
  };

  return (
    <div style={estiloTema}>
      <h3>Demostración de useMemo</h3>
      <p>Número base: <strong>{contador}</strong></p>
      <p>Resultado del cálculo:
    <strong>{resultadoCalculado}</strong></p>

      <button onClick={() => setContador(contador + 1)}>Incrementar
Número</button>
      <button onClick={() => setModoOscuro(!modoOscuro)}>Alternar Modo
Oscuro</button>
    </div>
  );
}

export default OptimizadorRendimiento;

```

Explicación del ejemplo:

1. La función calculoPesado contiene un bucle intencionalmente demandante.
2. Al envolverla en useMemo(..., [contador]), React almacena en caché el resultado.
3. Si el usuario pulsa "Alternar Modo Oscuro", el componente se re-renderiza para modificar los colores, pero React reutiliza el resultado previo de resultadoCalculado sin volver a ejecutar el bucle intensivo.

- Únicamente cuando el valor de contador cambia, React invalida la caché y recalcula la operación.

2. El Contexto en React (React Context)

¿Qué es el Contexto en React?

El **Contexto en React** (createContext y el hook useContext) es una API nativa que permite compartir y transmitir datos a través del árbol de componentes sin necesidad de pasar las propiedades (*props*) manualmente por cada uno de los niveles intermedios. Funciona como un almacén de datos global accesible para cualquier componente descendiente.

Características principales

- **Acceso descentralizado:** Cualquier componente hijo, sin importar cuán profundo esté en la jerarquía, puede suscribirse y leer los datos del contexto directamente.
- **Patrón Proveedor/Consumidor:** Utiliza un componente Provider (<Context.Provider value="{...}">) que envuelve la estructura de componentes para proveer los valores y estados, y el hook useContext para consumirlos.
- **Propagación reactiva:** Cuando el valor provisto en el Provider cambia, todos los componentes suscritos a dicho contexto se re-renderizan automáticamente con la nueva información.

¿Para qué sirve?

Sirve para almacenar datos "globales" que son requeridos por múltiples componentes dispersos en la aplicación, tales como:

- Información del usuario en sesión actual (autenticación).
- Preferencias visuales o tema de la interfaz (modo claro/modo oscuro).
- Configuración de idioma o internacionalización (i18n).
- Estado de un carrito de compras.

Diferencias entre Props y Context

Criterio	Props (Propiedades)	Context (Contexto)
Flujo de datos	Estricto de padre a hijo directo (nivel por nivel).	Transversal; viaja del Provider a cualquier hijo sin pasar por los intermedios.
Problema que resuelve / causa	Si la jerarquía es profunda causa Prop Drilling (pasar props por componentes que no las usan).	Elimina el <i>Prop Drilling</i> , desacoplando los componentes intermedios de los datos.

Alcance	Local y puntual entre dos componentes adyacentes.	Global o semiglobal para una rama entera de la aplicación.
Reutilización	Máxima; el componente es puro y no depende de un entorno externo.	Moderada; acopla el componente consumidor a la existencia de un Provider.

```
import React, { createContext, useContext, useState } from 'react';

//Creación del contexto global
const TemaContexto = createContext();

//Componente Nieto (Consumidor directo)
function BotonCambiarTema() {
  const { tema, alternarTema } = useContext(TemaContexto);

  return (
    <button
      onClick={alternarTema}
      style={{
        backgroundColor: tema === 'claro' ? '#eee' : '#333',
        color: tema === 'claro' ? '#000' : '#fff',
        padding: '10px 15px',
        border: 'none',
        borderRadius: '5px',
        cursor: 'pointer'
      }}
    >
      Tema actual: {tema.toUpperCase()} (Clic para alternar)
    </button>
  );
}

// Componente Hijo Intermedio
function BarraNavegacion() {
  return (
    <header style={{ borderBottom: '1px solid #ccc', padding: '10px'
  }}>
```

```

    <h2>Mi Aplicación</h2>
    <BotonCambiarTema />
  </header>
);
}

// Componente Padre (Proveedor)
function Aplicacion() {
  const [tema, setTema] = useState('claro');

  const alternarTema = () => {
    setTema((temaActual) => (temaActual === 'claro' ? 'oscuro' :
'claro'));
  };

  return (
    <TemaContexto.Provider value={{ tema, alternarTema }}>
      <div style={{ minHeight: '150px', padding: '20px' }}>
        <BarraNavegacion />
      </div>
    </TemaContexto.Provider>
  );
}

export default Aplicacion;

```

Explicación del ejemplo:

1. `createContext()` inicializa la vía de comunicación `TemaContexto`.
2. El componente raíz `Aplicacion` utiliza `TemaContexto.Provider` para poner a disposición el estado `tema` y la función `alternarTema`.
3. El componente intermedio `BarraNavegacion` no recibe ni conoce ninguna propiedad relacionada al tema, eliminando el problema de *prop drilling*.
4. `BotonCambiarTema`, situado en el nivel más bajo, invoca `useContext(TemaContexto)` para leer el estado y ejecutar la función actualizadora directamente.